

Natural Language Processing

8. Seq2seq, CNN, and transformers

Paolo Torrioni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Sequence to Sequence Networks
- Convolutional Networks for NLP
- Transformers

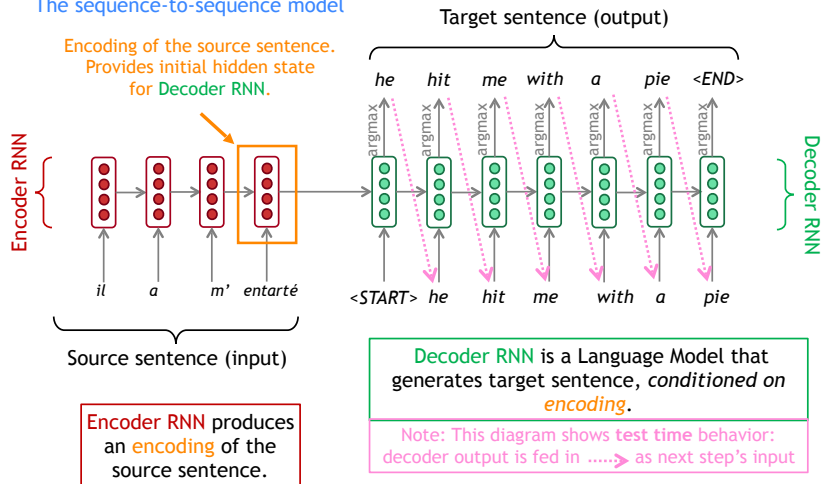
Sequence to Sequence Networks

- A **Sequence to Sequence** network, or seq2seq network, or Encoder Decoder network, is a model consisting of two RNNs called the encoder and decoder.
- The encoder reads an input sequence and outputs a single vector, and the decoder reads that vector to produce an output sequence.

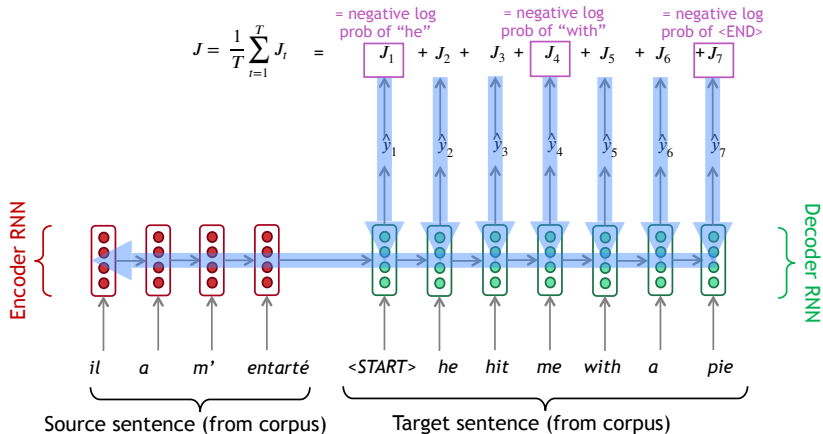
Pytorch tutorial: [NLP From Scratch: Translation with a Sequence to Sequence Network and Attention](#)

The Seq2seq Model

The sequence-to-sequence model

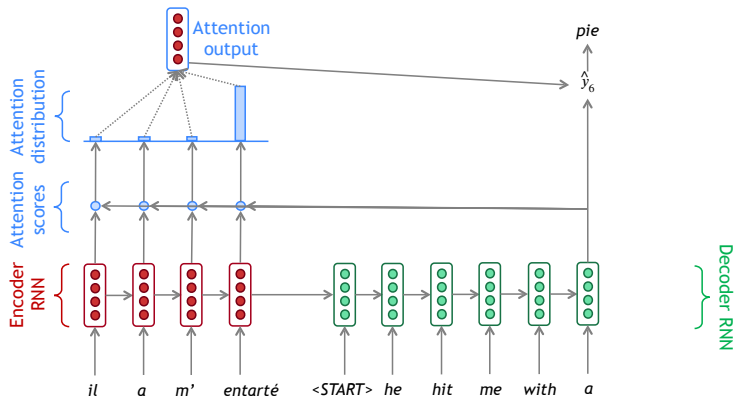


Training Seq2seq



RNN Encoder-Decoder with Attention

- Consider a **sequence-to-sequence** architecture for MT
- On each step of the decoder, use **direct connection to the encoder** to **focus on a particular part** of the source sequence

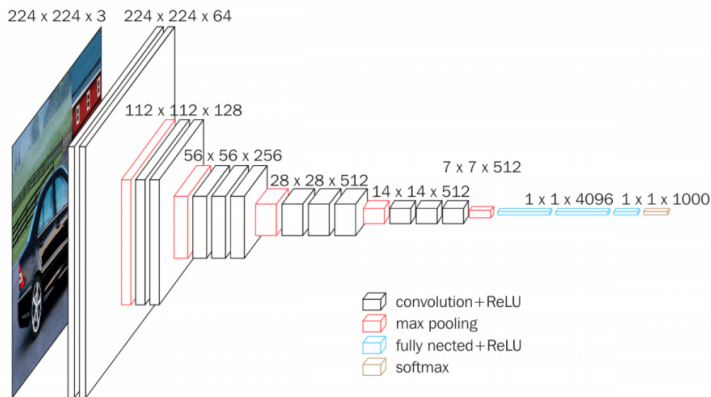


RNN Encoder-Decoder with Attention

- **Encoder** hidden states: $h_1, \dots, h_N \in \mathbb{R}^h$
- Hidden state of **decoder** at timestep t : s_t
- **Attention scores** at timestep t $e^t = [s_t^\top h_1, \dots, s_t^\top h_N] \in \mathbb{R}^N$
- **Attention distribution** α^t used to take a weighted sum of the encoder hidden states: $\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$
- **Attention output** mostly contains information from the hidden states that received high attention $a_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$
- **Concatenate attention output with decoder hidden state:**
 $[a_t; s_t] \in \mathbb{R}^{2h}$

Convolutional Networks for NLP

How *Deep* Networks for CV such as VGG-16 Look Like



- Could we have the same type of networks for text processing?

From RNNs to ConvNets

- **1D convolution** idea: compute vectors for every possible word subsequence of a certain length
- Example: “*tentative deal reached to keep government open*” computes vectors for:
 - tentative deal reached
 - deal reached to
 - reached to keep
 - to keep government
 - keep government openregardless of whether phrase is grammatical
- Not very linguistically or cognitively plausible
- Then group them afterwards

1D Convolution Example

tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3

t,d,r	-1.0
d,r,t	-0.5
r,t,k	-3.6
t,k,g	-0.2
k,g,o	0.3

- Apply a **filter** (or **kernel**) of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1D Convolution Example

∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6
t,d,r	-1.0
d,r,t	-0.5
r,t,k	-3.6
t,k,g	-0.2
k,g,o	0.3
g,o,∅	-0.5

- Apply a **filter** (or **kernel**) of size 3, (zero) padding = 1
 - could also use padding = 2, *wide convolution*

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1D Convolution Example

∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6	0.2	1.4
t,d,r	-1.0	1.6	-1.0
d,r,t	-0.5	-0.1	0.8
r,t,k	-3.6	0.3	0.3
t,k,g	-0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,∅	-0.5	-0.9	0.1

- Apply **three filters** of size 3, (zero) padding = 1

3	1	2	-3	1	0	0	1	1	-1	2	-1
-1	2	1	-3	1	0	-1	-1	1	0	-1	3
1	1	-1	1	0	1	0	1	0	2	2	1

1D Convolution Example

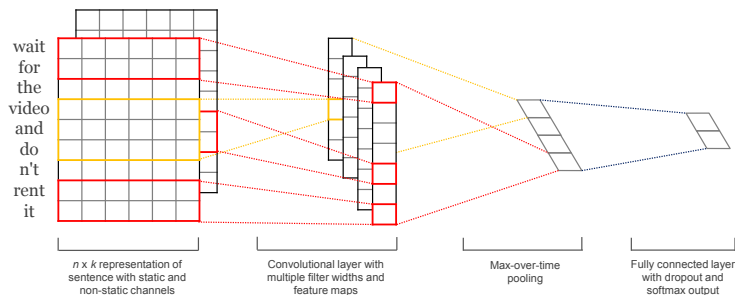
∅	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
∅	0.0	0.0	0.0	0.0

∅,t,d	-0.6	0.2	1.4
t,d,r	-1.0	1.6	-1.0
d,r,t	-0.5	-0.1	0.8
r,t,k	-3.6	0.3	0.3
t,k,g	-0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,∅	-0.5	-0.9	0.1

max p	0.3	1.6	1.4
-------	-----	-----	-----

- Apply **three filters** of size 3, (zero) padding = 1
- **Max pooling over time**
 - Alternatively, **average** pooling over time
- Could also play with stride, dilation, etc.
- Also interesting size 1 convolutions (**network-in-network**)

ConvNets for Sentence Classification



- Sentence is concatenation of n k -dimensional word vectors
- Convolutional filter $w \in \mathbb{R}^{hk}$ applied to a window of h words to produce a new feature map $\in \mathbb{R}^{n-h+1}$
- **Max-over-time pooling** deals with variable sentence lengths
- Word vectors may be fine-tuned as well as kept static (multi-channel)

Convolutional Neural Networks for Sentence Classification, Y Kim, EMNLP 2014

Character-Aware Neural Language Models

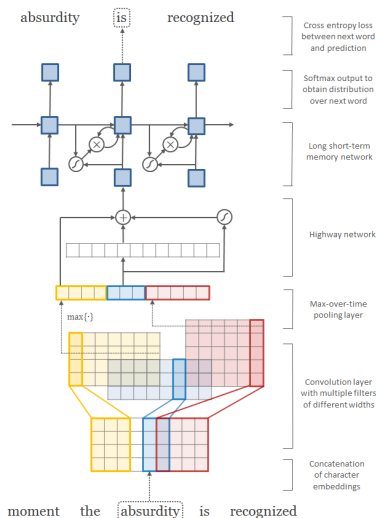
An RNN language model that relies only on character-level inputs

- In this illustration, character embeddings of size 4, then
 - four filters of width 3 (yellow)
 - three of width 2 (blue)
 - five of width 4 (red)

then max-pooling for fixed-dimensional word representation

- Code available
- Significant model-size reduction and state of the art results

Character-Aware Neural Language Models, Y Kim et al, AAAI 2016



Comparing Models for Sentence Encoding

- **Bag of Vectors:** Surprisingly good baseline for simple classification problems. Especially if followed by a few ReLU layers!
- **Window Model:** Good for single word classification for problems that do not need wide context. E.g., POS, NER
- **CNNs:** good for classification, need zero padding for shorter phrases, somewhat implausible/hard to interpret, **easy to parallelize on GPUs**. Efficient and versatile
- **RNNs:** Cognitively plausible (reading from left to right), not best for classification (if just use last state), much slower than CNNs, good for sequence tagging and classification, great for language models, even better with **attention** mechanisms

Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, Reimers & Gurevych, EMNLP 2019

Transformers

Pretrained Language Models

- Seen so far: embeddings learned by **self-supervision** can be used as they are (without updating them) for other tasks
- Idea of **pretraining**: learning knowledge about language and the world from vast amounts of text
- **Large language models** are “big” pretrained models (many parameters) that can be used as a **foundation** for downstream applications
- A limitation of the neural language model seen earlier is the limited context window. **RNNs** overcome that problem, but suffer from vanishing gradients, and take time to train
- The **transformer** is the standard architecture for building large language models, that is effective at handling **distant information** and supports parallel processing

Transformer: Overview

- Transformer components: **input encoding, blocks, LM head**

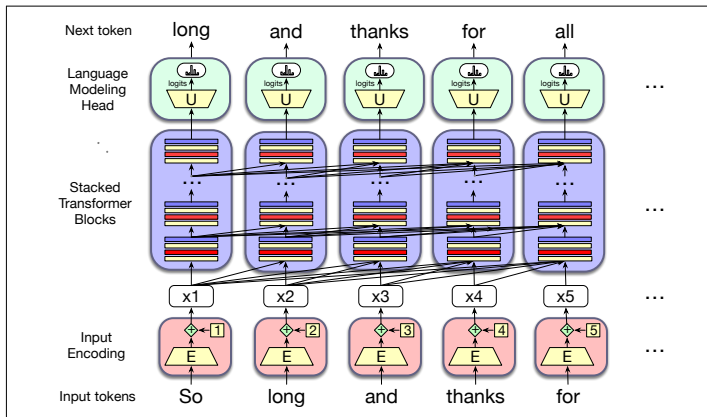


Figure 9.1 The architecture of a (left-to-right) transformer, showing how each input token get encoded, passed through a set of stacked transformer blocks, and then a language model head that predicts the next token.

Transformer: Overview

- The **transformer** is a neural architecture made up of stacks of transformer **blocks**
- Each block is a multilayer network that maps sequences of input vectors $(x_1, \dots, x_n)$ of **dimensionality** d to sequences of output vectors $(z_1, \dots, z_n)$ of the same length and dimensionality

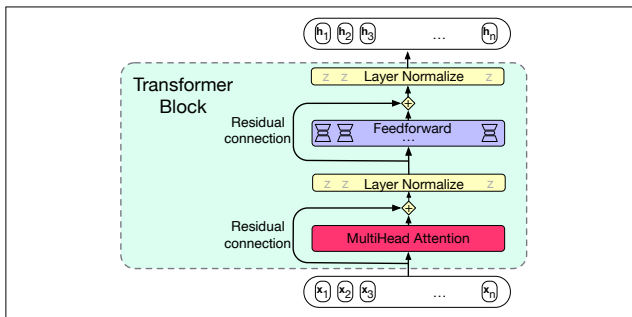
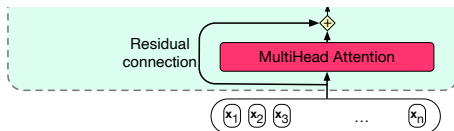


Figure 10.6 A transformer block showing all the layers.

Transformer Block



- Each block contains a **self-attention** layer, the key innovation of transformers
- Self-attention allows a network to directly extract and use information from arbitrarily large contexts
- At each layer of a transformer (block), to compute the representation of a word i we combine information from the representation of i at the previous layer with information from the representations of the neighboring words
- The goal is to obtain a **contextualized representation**, representing what a word means in the particular context in which it occurs

Self-attention

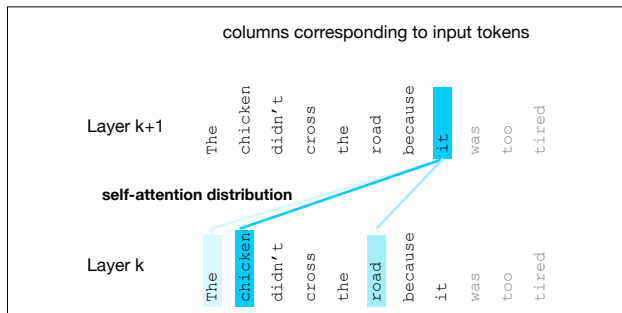


Figure 9.2 The self-attention weight distribution α that is part of the computation of the representation for the word *it* at layer $k + 1$. In computing the representation for *it*, we attend differently to the various words at layer l , with darker shades indicating higher self-attention values. Note that the transformer is attending highly to the columns corresponding to the tokens *chicken* and *road*, a sensible result, since at the point where *it* occurs, it could plausibly corefers with the chicken or the road, and hence we’d like the representation for *it* to draw on the representation for these earlier words. Figure adapted from [Uszkoreit \(2017\)](#).

- Notice: if the aim is language modeling, the context to consider is only that to the left of the word to predict (**causal attention**)

Self-attention

- The core idea is to **compare** pairwise all elements of the input sequence, and emphasize their **relevance** to one another
- The result of these comparisons is used to compute an output sequence for the current input sequence
- For example, the computation of a_3 is based on a set of comparisons between the input x_3 and its preceding elements x_1 and x_2 and itself

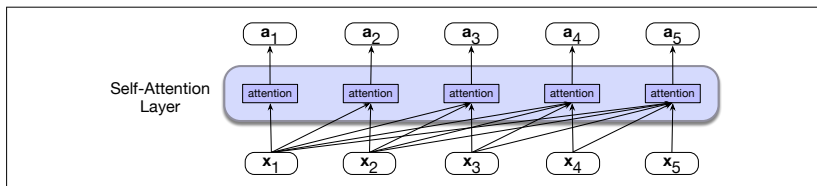


Figure 9.3 Information flow in causal self-attention. When processing each input x_i , the model attends to all the inputs up to, and including x_i .

Self-attention

- How to measure the similarity between vectors?

👉 Seen already with seq2seq models...**dot product**!

- The dot product between two vectors is a scalar (**score**)
- Softmax can be used to normalize the scores

$$\alpha_{ij} = \frac{e^{\mathbf{x}_i \cdot \mathbf{x}_j}}{\sum_{k=1}^i e^{\mathbf{x}_i \cdot \mathbf{x}_k}} \forall j \leq i$$

- Such scores can be used to generate an output vector as an average of all the input vectors, weighted by their relevance

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

Self-attention

- However, we need to distinguish between three different **roles** each input embedding $\mathbf{x}_i$ plays
 - as *the current focus of attention* when being compared to all other inputs (**query**)
 - as *the preceding input* being compared to the current focus of attention (**key**)
 - as *the value used to compute the output* for the current focus of attention (**value**)
- To capture these three roles, transformers create three linear **projections** of each input embedding $\mathbf{x}_i$ of size d , using three different (learnable) weight matrices:

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

- Usually, key and query embeddings have the same dimensionality d_k , value embeddings have dimensionality d_v

Self-attention

- In this way, for comparing inputs, we use query and key

$$\mathbf{q}_i \cdot \mathbf{k}_j$$

- To prevent arbitrarily large scores, we divide the result by a factor related to the size d_k of the embeddings. We mask out the “future”

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \quad \forall j \leq i; \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) = 0 \quad \forall j > i$$

- We normalize using softmax, obtaining attention weights

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))$$

- We obtain the output for a **head** from a weighted sum of the **values**, through a reshaping matrix $\mathbf{W}^O \in \mathbb{R}^{d_v \times d}$:

$$\text{head}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j; \quad \mathbf{a}_i = \text{head}_i \mathbf{W}^O$$

Self-attention

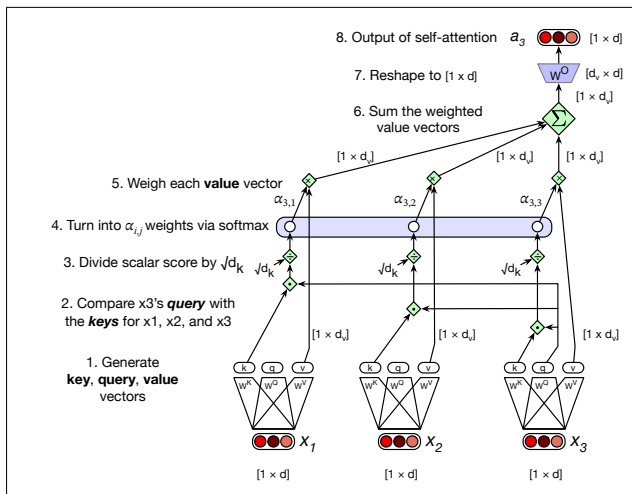


Figure 8.4 Calculating the value of a_3 , the third element of a sequence using causal (left-to-right) self-attention.

Multi-Head Attention

- The different words in a sentence can relate to each other in many different ways simultaneously
- A single self-attention model can hardly learn all these kinds of parallel relations

 Transformers introduce **multihead self-attention layers**

Multi-Head Attention

- The model dimension d is still used for input and output, and d_k , d_v for the dimension of query/key and value projections
- However, for each head $c \in \{1, \dots, A\}$ we have
 - $\mathbf{W}^{Qc} \in \mathbb{R}^{d \times d_k}$
 - $\mathbf{W}^{Kc} \in \mathbb{R}^{d \times d_k}$,
 - $\mathbf{W}^{Vc} \in \mathbb{R}^{d \times d_v}$
- We end up with A different outputs of size d_v
- These results are concatenated into a single output vector of size $h \times A d_v$, reshaped to size d using a $\mathbf{W}^O \in \mathbb{R}^{A d_v \times d}$

Multi-Head Attention

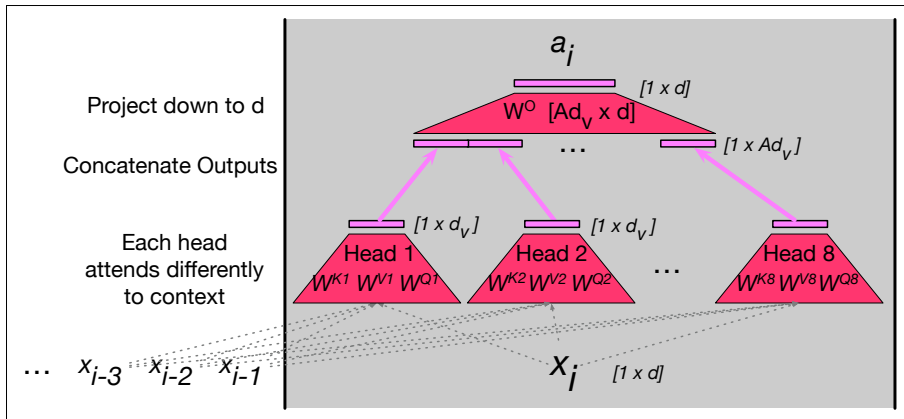


Figure 8.5 The multi-head attention computation for input x_i , producing output a_i . A multi-head attention layer has A heads, each with its own query, key, and value weight matrices. The outputs from each of the heads are concatenated and then projected down to d , thus producing an output of the same size as the input.

Multi-Head Attention

- In summary, $\mathbf{a}_i \in \mathbb{R}^d = \text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_{i-1}])$ where:

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{Qc}; \quad \mathbf{k}_i^c = \mathbf{x}_i \mathbf{W}^{Kc}; \quad \mathbf{v}_i^c = \mathbf{x}_i \mathbf{W}^{Vc} \quad \forall 1 \leq c \leq A$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}} \quad \forall j \leq i; \quad \text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = 0 \quad \forall j > i$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j))$$

$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\text{head}_i^1 \oplus \text{head}_i^2 \oplus \dots \oplus \text{head}_i^n) \mathbf{W}^O$$

Transformer Block

The transformer block contains other layers besides the Multi-Head Attention layer

- A **feedforward layer** with N position-wise networks, one at each position. Each is a fully-connected 2-layer network, i.e., one hidden layer, and two weight matrices. The weights are the same for each position, but the parameters are different from layer to layer.
- **Residual connections** to help propagate the information during training
- **Normalization layers** to improve training performance by keeping the values of a hidden layer in a range that facilitates gradient-based training
- Crucially, the input and output dimensions of transformer blocks are matched so they can be stacked.

The Residual Stream

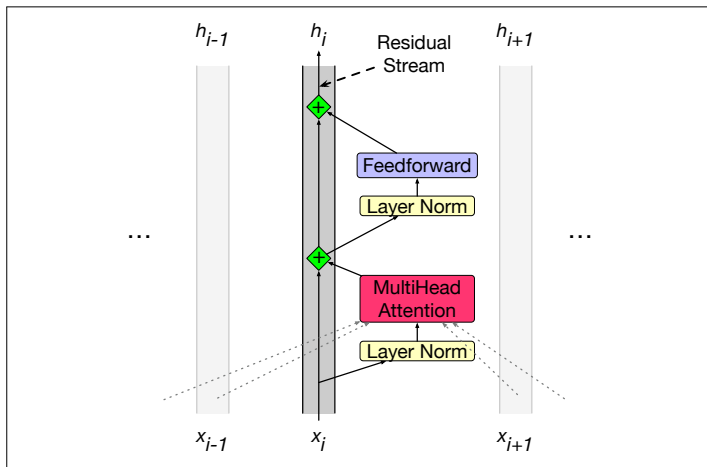


Figure 9.6 The architecture of a transformer block showing the **residual stream**. This figure shows the **prenorm** version of the architecture, in which the layer norms happen before the attention and feedforward layers rather than after.

The Residual Stream

- The **residual stream** is an intuitive way of thinking about the transformer block, where an individual token i is processed through a single stream of d -dimensional representations
- The FF layer is a 2-layer network (one hidden layer), where
 - the weights are the **same for each token position** i
 - but are different from layer to layer: $\mathbf{W}_1, \mathbf{W}_2, \dots$

$$FFN(x_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$

- The dimensionality d_{ff} is usually greater than d
- Normalization is crucial to facilitate gradient-based training
 - it is not applied to an entire transformation layer, but just to the embedding vector of **a single token** of dimension d
 - it produces a new embedding with zero mean and a unit standard deviation

Input Embedding

- The input is tokenized using BPE, SentencePiece or similar
- To encode each token, we resort to the same method seen for neural language modeling, with an **embedding matrix E**.

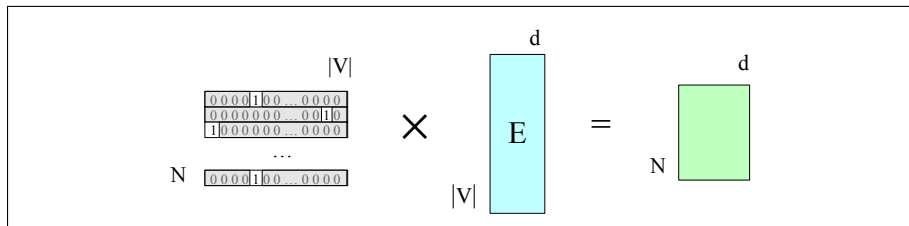


Figure 9.12 Selecting the embedding matrix for the input sequence of token ids W by multiplying a one-hot matrix corresponding to W by the embedding matrix E .

Positional Encoding

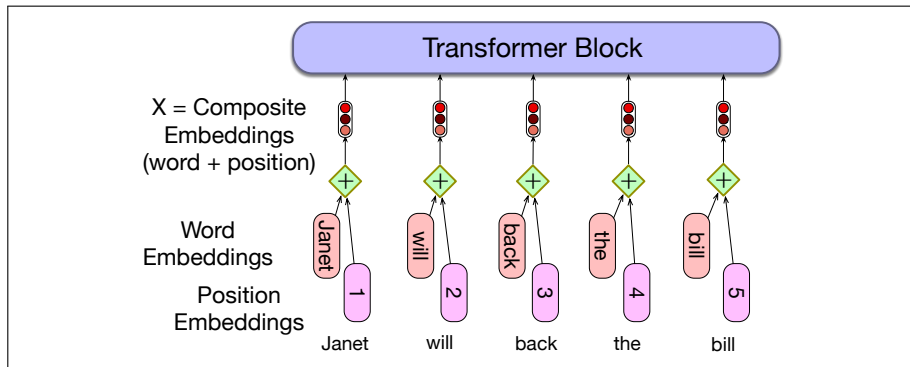


Figure 10.12 A simple way to model position: add an embedding of the absolute position to the token embedding to produce a new embedding of the same dimensionality.

The Language Modeling Head

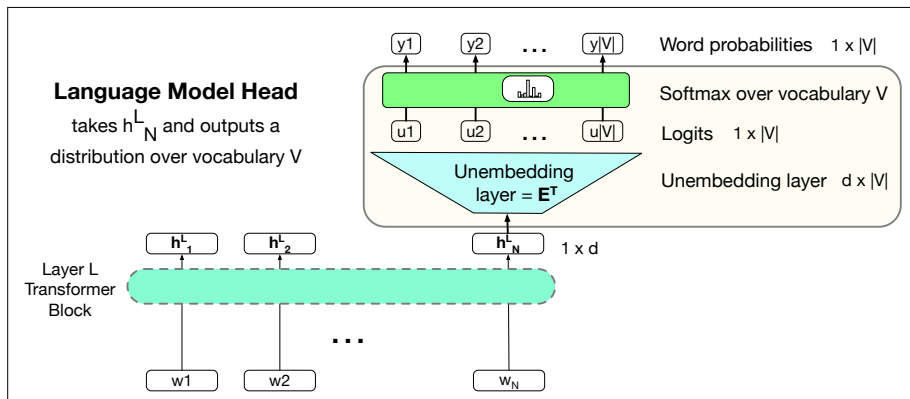


Figure 10.13 The language modeling head: the circuit at the top of a transformer that maps from the output embedding for token N from the last transformer layer (h_N^L) to a probability distribution over words in the vocabulary V .

Putting It All Together

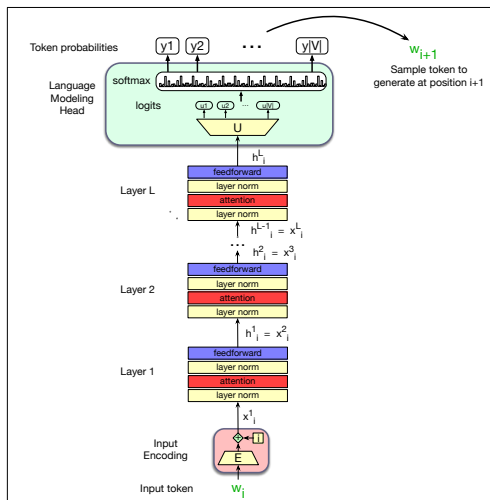


Figure 9.15 A transformer language model (decoder-only), stacking transformer blocks and mapping from an input token w_i to a predicted next token w_{i+1} .

Questions?